

Simulación del ciclo de ejecución de instrucciones

Fernando Vicente Muñoz
Benemérita Universidad Autónoma de
Aguascalientes
Departamento de Sistemas Electrónicos
Aguascalientes, México
al575994@edu.uaa.mx

Abstract— Durante esta práctica se simuló el ciclo de ejecución de instrucciones utilizando un sistema binario de 9 bits, organizado en tres grupos de 3 bits. A partir de esta representación se buscó comprender de forma simplificada el funcionamiento del *pipeline*, considerando cuatro etapas principales: Fetch, Decode, Execute y Memory Writeback. Para llevar a cabo la simulación se siguieron algoritmos previamente establecidos, los cuales permitieron organizar el uso de los espacios de memoria y representar el recorrido de cada instrucción. De esta manera, la práctica permitió visualizar de una forma más sencilla el proceso que sigue una instrucción desde que es obtenida hasta que finaliza su ejecución dentro del procesador.

Keywords— CPU, instrucciones binarias, ciclo de instrucción, pipeline, memoria, Fetch, Decode, Execute, Writeback.

Introducción

El procesador es el encargado de ejecutar las instrucciones que forman parte de un programa. Para realizar esta tarea, cada instrucción atraviesa un proceso conocido como ciclo de instrucción, en el cual la CPU obtiene la instrucción desde la memoria, determina la operación que debe realizar y finalmente la ejecuta. De manera general, este proceso puede dividirse en las etapas de Fetch, Decode y Execute, aunque también puede considerarse una cuarta etapa denominada Write-Back, encargada de escribir el resultado obtenido durante la ejecución [1].

Comprender estas etapas permite observar de una manera más sencilla lo que sucede dentro de un procesador durante la ejecución de un programa. Por esta razón, en la presente práctica se realiza una representación simplificada de este proceso mediante instrucciones binarias, con el propósito de analizar el recorrido de una instrucción y comprender la función que cumple cada una de las etapas involucradas.

I. FUNCIONAMIENTO DEL SISTEMA DE INSTRUCCIONES DE 9 BITS

Para esta práctica se utilizó un sistema de instrucciones binarias de 9 bits con el propósito de representar de manera simplificada el funcionamiento de un procesador. Cada instrucción se encuentra dividida en tres bloques de 3 bits. El primer bloque indica la operación que debe realizarse, mientras que los otros dos corresponden a los operandos que participan en dicha operación. Al estar formados por 3 bits, los operandos pueden representar valores desde 000 hasta 111, equivalentes a los números decimales del 0 al 7. [2]

Para identificar las operaciones se establecieron diferentes códigos binarios. El código 000 corresponde a la suma, 001 a la resta, 010 al incremento, 011 al decremento y 100 al movimiento de información entre memorias. De esta forma, al recibir una instrucción de 9 bits, el sistema primero identifica los tres bits iniciales para conocer la operación que debe realizar y posteriormente utiliza los siguientes bloques para determinar los operandos involucrados.[2]

Una característica importante del sistema es el uso de la Memoria 1 como un espacio temporal para almacenar los resultados obtenidos en las operaciones aritméticas. Cuando se realiza una suma, resta, incremento o decremento, el resultado no se coloca directamente en la memoria de destino, sino que primero se guarda en la Memoria 1. Posteriormente, mediante una instrucción de movimiento de memoria, este resultado es transferido hacia la posición que haya sido indicada dentro del algoritmo.[2]

Por ejemplo, si se realiza una suma entre dos valores, el procesador identifica los operandos correspondientes y efectúa la operación. El resultado obtenido se almacena inicialmente en la Memoria 1. Después se utiliza la operación de movimiento, representada por el código 100, para trasladar ese resultado desde la Memoria 1 hasta la memoria en la que se requiere almacenarlo. De esta manera, la Memoria 1 funciona como un espacio de trabajo temporal que permite conservar los resultados antes de colocarlos en su destino final.[2]

Este sistema permite comprender de una manera práctica cómo un procesador sigue una serie de instrucciones para ejecutar un algoritmo. Cada instrucción determina la operación que se realizará y los datos que se utilizarán, mientras que las memorias permiten conservar y trasladar la información durante el proceso. Así, mediante una secuencia ordenada de operaciones aritméticas y movimientos de memoria, es posible representar de forma sencilla la manera en que un procesador interpreta y ejecuta diferentes instrucciones.[2]

II. RESULTADOS

La Tabla I presenta las instrucciones que forman parte de la simulación de ejecución, detallando además los desplazamientos requeridos para guardar cada resultado en su respectiva dirección de memoria. Por su parte, la columna denominada Fetch de esa misma tabla expone el código máquina derivado del algoritmo, al tiempo que las columnas restantes detallan las fases de las que consta el ciclo de ejecución para cada instrucción.

ALGORITMO:

- Memoria 2 = 2 + 2
- Memoria 3 = 5 + 1
- Memoria 4 = Memoria 2
- Memoria 5 = 2 - 1

TABLA 1. ALGORITMO APLICADO A ISA DE 9 BITS

Fetch	Decode	Execute	Memory / Writeback
000 010 010	Suma 2 + 2	2 + 2 = 4	001(1)
100 001 010	Mover M1 → M2	M1 → M2	010(2)
000 101 001	Suma 5 + 1	5 + 1 = 6	001(1)
100 001 011	Mover M1 → M3	M1 → M3	011(3)
100 010 100	Mover M2 → M4	M2 → M4	100(4)
001 010 001	Resta 2 - 1	2 - 1 = 1	001(1)
100 001 101	Mover M1 → M5	M1 → M5	101(5)

Al finalizar la simulación, se registraron los valores de cuatro en la Memoria 2, seis en la Memoria 3, cuatro en la Memoria 4 y uno en la Memoria 5. Las operaciones comenzaron empleando la Memoria 1 como un depósito provisional para los datos, los cuales fueron posteriormente

trasladados a sus ubicaciones finales por medio de instrucciones de transferencia de acuerdo con el algoritmo.

III. CONCLUSIONES

La práctica ayudó a comprender cómo se ejecutan las instrucciones y qué fases ocurren durante ese proceso. Mediante el uso de código binario fue posible representar sumas, restas y movimientos entre distintos espacios de memoria, viendo cómo cada instrucción pasa por las etapas de búsqueda, decodificación, ejecución y almacenamiento.

El uso de algoritmos demostró que los resultados no se guardan en orden lineal, sino que deben enviarse a lugares específicos de memoria mediante movimientos. Además, la simulación permitió representar de forma sencilla el funcionamiento de un pipeline, facilitando entender cómo un procesador organiza y ejecuta sus tareas paso a paso.

REFERENCES

- [1] HardZone, “Ciclo de instrucción de una CPU: cómo funciona y sus etapas,” HardZone. [Online]. Available: <https://hardzone.es/tutoriales/rendimiento/ciclo-instruccion-cpu/>. [Accessed: Aug. 30, 2026].
- [2] OpenAI, ChatGPT, respuesta generada al prompt “Explicame de forma didáctica, para un reporte, cómo funciona un procesador básico que ejecuta instrucciones de 9 bits, divididas en tres bloques de 3 bits: el primero para la operación (como suma 000, resta 001, incremento 010, decremento 011 y movimiento de memoria 100), y los otros dos bloques como operandos usando valores binarios del 0 al 7. Tienes que saber que los resultados aritméticos se guardan en Memoria 1 para así después ser pasados a donde se pidió”, Aug. 30, 2026.